Adversarial Gameplay Report: Defending Against My Own Attack on AI-Image Detectors

Yunjie Wang, Student ID 36711772, Team Dark.ModelBreakers (Theme 1, Dark side)

***Abstract*—This report is the Milestone 4 individual submission for the FIT5230 team project. Our team took the Dark side of Theme 1 and improved StealthDiffusion, an attack that hides AI-generated images from forensic detectors. The report has four parts. First, a weekly log of the strategies I used from Week 2 to Week 12. Second, what I would do to improve the attack if there were no time or resource limits: move the detector objective from per-image optimisation into the training of the Control-VAE control branch. A small demo (2 epochs, 150 images, 80 s) already makes the plain reconstruction evade three of the four detectors. Third, the role reversal: if I had to defend against our own attack, what I would do. I show that a defender who keeps the released detectors and only re-calibrates the decision interval on real photographs catches 99% of our attacked images at a 4% false positive rate, without any retraining or pre-processing. Fourth, a personal reflection on how my background shaped the work. All numbers in the defence section are recomputed from the archived Milestone 3 measurements by the scripts that accompany this report.**

***Index Terms*—adversarial examples, AI-generated image detection, StealthDiffusion, diffusion models, adversarial defence.**

I. Introduction

OUR team, Dark.ModelBreakers, chose Theme 1 (AI-generated image detection) on the Dark side. The attack we worked on is StealthDiffusion [1], which takes an image made by Stable Diffusion and changes it so that forensic detectors call it a real photograph. The pipeline has three parts: a pixel-space PGD step [2], a latent adversarial optimisation (LAO) inside the Stable Diffusion latent space, and a modified VAE decoder (Control-VAE).

Over Milestones 1 to 3 our team reproduced the released code, added switches for several improvements, built an evaluation protocol that the original paper does not have (conditional attack success rate with a stated denominator, real-image false positive rate, AUC, JPEG and resize robustness), and ran the attack on 100 SD 1.5 images against the four detectors released with the paper (EfficientNet E, ResNet R, DeiT D, Swin S). Three findings from that work matter for this report:

1) In the released code the LAO step barely reaches the saved image. Removing it changes 4 of 400 detector decisions. The attack that reaches the file is a pixel-space PGD perturbation.
2) The attack *overshoots*. On detector E the attacked images get a real-vs-AI margin of about $+150$, while real photographs sit between 0 and $+47$. The AUC between real and attacked images is 0.00, i.e. they are perfectly separable, only in the wrong direction.
3) The ensemble of surrogate detectors (ENS) is the only one of our five switches that changes decisions. It raises the transfer success rate on R and D from $0.77/0.39$ to $1.00/1.00$.

Section II is the weekly log. Section III is the unconstrained enhancement. Section IV is the role reversal. Section V is the reflection.

II. Weekly Strategy Log (Weeks 2–12)

The log is written from my own records (dated project notes, git commits and submission times). Weeks are counted from the unit calendar (Week 5 ends with the Milestone 1 deadline, 28 August; the mid-semester break falls between Weeks 8 and 9). One short paragraph per week: what I did, why, and whether it paid off.

Week 2 (3–9 Aug). Read the assignment with my teammate and mapped where the marks are: Milestone 3 and 4 together carry the peer engagement and the individual items, so an early decision on theme and paper would buy time later. We chose Theme 1 (AI-image detection) on the Dark side, because a released attack code base gives a concrete thing to improve and measure.

Week 3 (10–16 Aug). Shortlisted candidate papers with my teammate and weighed two things for each: how hard it is to reproduce, and how much room it leaves for a real improvement. Ran partial baselines of a few of them to see which code actually works.

Week 4 (17–23 Aug). Settled on StealthDiffusion [1]: released code, released detector checkpoints, and a method with three separable parts, which means ablations are possible. Ran the official repository and obtained a first baseline.

Week 5 (24–30 Aug). Finished and submitted Milestone 1 (28 Aug): the official pipeline on six images, a small synthetic experiment on frequency-domain perturbations, and a challenge set for other teams. The free Colab T4 ran out of memory, so the full run was done on a local GPU; this was the first sign that local hardware would be my main advantage. On 29 Aug I wrote a technical document for the team, so that the rest of the semester did not depend on re-reading the paper.

Week 6 (31 Aug–6 Sep). Read the Milestone 2 requirements and discussed the plan with my teammate: which of the paper's weaknesses to attack first, and how to split the work.

Week 7 (7–13 Sep). Searched for other papers that use similar strategies (latent-space attacks, attacks on forensic detectors, post-processing robustness), to see what a reviewer

FIT5230 Malicious AI, Monash University Malaysia, Semester 2, 2026. Milestone 4 individual report.

would compare us against. This became the related-work base for the paper draft in Week 10.

Week 8 (14–20 Sep). Milestone 2 week. The key strategic decision, written down on 14 Sep: *Milestone 2 measures, Milestone 3 fixes*. We deliberately held improvements back so that Milestone 3 would have real work left that follows from our own Milestone 2 results, instead of a new idea invented at the last minute. I did the code: five configuration switches and four variants, a full evaluation protocol (conditional ASR with a stated denominator, real-image FPR, AUC, JPEG and resize), a BatchNorm training-mode bug in the ensemble wrapper found and fixed, and two negative results recorded (the latent optimisation flips no decision; the paper's weakness (b) does not occur). My teammate integrated the scripts into the Colab notebook and ran it, and wrote the Ed post (17 Sep) and the first messages to three Light teams (PreDecodeGuard, Umbrella, Fahhh); I wrote a handover document and a list of reserved Milestone 3/4 items so that both of us knew what not to touch yet. Submitted on 18 Sep.

Week 9 (28 Sep–4 Oct). Planned Milestone 3 as an experiment list rather than a feature list: which detectors, which post-processing levels (JPEG 95/75/50, resize 0.75/0.5), which metrics, and three hypotheses about which component of the pipeline does the work. Decided that every rate would be recomputed from saved files and that hypotheses would be frozen before running. Organised the Week 7 survey into a related-work section.

Week 10 (5–11 Oct). Wrote the full write-up of the code audit and the evaluation protocol (two versions) and ran two rounds of hostile review against it. The review found that in the released code the latent optimisation computes its loss on a decoder path that is never saved; this became Hypothesis 1 and the backbone of Milestone 3. It also caught two factual errors in my own first version, which is why I kept the review step.

Week 11 (12–18 Oct). Froze the hypotheses as a git tag before running anything, then ran the whole Milestone 3 experiment set locally on the M5 Max (SD 1.5, 100 images, 20+ configurations), as resumable overnight batches, instead of waiting for Colab. Results: the latent optimisation changes 4 of 400 decisions, the attack lives in the pixel PGD step, an 8/255 pixel PGD matches the full pipeline, and a capped and an EOT variant each fix one weakness. Ran three further LAO architecture experiments. Received the PreDecodeGuard pilot package and declined the part of it that would have produced images bypassing a safety classifier on violent content; proposed three analyses that did not, and wrote the decline into the engagement record. Two handover documents to my teammate (17 and 18 Oct).

Week 12 (19–25 Oct). Produced the Colab notebook with pre-rendered outputs (verified run on 21 Oct), wrote the Milestone 3 engagement posts, submitted Milestone 3 on 22 Oct with the five-minute individual talk, and added the Milestone 2-to-3 introduction on 23 Oct.

What the log shows. The work was front-loaded in Weeks 2–5 and Week 8, thin in Weeks 6–7 and 9, and very dense in Weeks 10–12. The strategy that made the dense weeks possible was decided early: reserve the Milestone 3 topics in Week 8, fix the experiment list and protocol in Weeks 9–10 before touching the GPU, and keep every experiment on a local, resumable script so that a week of GPU time could be spent in three days. The advantage over other teams was the measurement protocol; the advantage over my teammate was that the code base, the evaluation scripts and the paper draft lived on my machine and in my notes, which is also why I wrote the handover documents.

III. Unconstrained System Enhancement and Feasibility

A. The idea: move the attack from test time to training time

Everything our team did in Milestones 2 and 3 happens *per image*: two PGD calls and a latent optimisation, about 10 s per image on a laptop GPU and a few seconds on an A100. The StealthDiffusion paper itself describes a different lever that we never touched: the Control-VAE is *trained* so that its output carries the noise statistics of real photographs (the "noise prototype" loss). I read the released training script, `train_vae.py`, to see how this is done. It is not. The script trains the control encoder on real photographs with LPIPS + L1 + a frequency loss; the noise prototype is computed and saved, but the variable that would add it to the loss is set to zero and never updated. No detector appears anywhere in training.

So the enhancement is: put the detector objective into the training of the control branch, once, and then launder every generated image with a single encode–decode pass. With no time or resource limits I would train on GenImage [3] (millions of images from eight generators), against an ensemble of all detectors I can obtain including CLIP-based ones [4], with JPEG and resize views inside the loss [5]. The product would be a decoder, not an attack: anyone who decodes through it gets an image the detectors call real, at the cost of one forward pass.

B. Why it would be a significant improvement

Three reasons. (i) Cost: the per-image part of the pipeline disappears, so the attack scales to any number of images at decoder speed. (ii) No overshoot by construction: training uses a margin loss that stops at $\kappa = 3$, so the output lands *inside* the real-image score range instead of far above it, which is exactly the weakness Section IV exploits. (iii) It is the only part of the original method we have not yet audited. Milestone 3 showed that the test-time latent optimisation contributes almost nothing, and a further Milestone 3 experiment that combined latent optimisation, PGD and EOT on 50 new images beat PGD+EOT by only 5/33 to 3/33 under four conditions (McNemar $p = 0.69$, not significant) at $1.9\times$ the distortion and $4.4\times$ the time. The test-time lever is exhausted; if the method has a real contribution left, it is at training time.

C. Feasibility demo

The demo (`m4_finetune_cvae.py`, driven by `m4_run.sh`) fine-tunes only the control encoder (38.4 M

parameters: the SD encoder copy and the four zero-convolutions that feed the skip connections). The SD VAE, the skip decoder and the detectors are frozen; detectors are kept in eval mode (the BatchNorm pitfall we found in Milestone 2). The training proxy for the final decode path is $z = 0.18215 \cdot \mathrm{enc}(x)$, $\hat{x} = \mathrm{Dec}(\mathrm{post_quant}(z/0.1825), \mathrm{CtrlEnc}(x))$, the same two scale constants the released code uses. The loss is

$$\mathcal{L} = \|x - \hat{x}\|_1 + \mathrm{LPIPS}(x, \hat{x}) + 0.05 \cdot \tfrac{1}{3} \sum_{d \in \{E,R,D\}} \max\big(0,\ \kappa - m_d(\hat{x})\big),$$

with $\kappa = 3$. Training data are the 150 SD 1.5 images generated for the Milestone 3 validation batches, which are disjoint from the 100-image test set; the 10 development images are used only to pick the checkpoint. AdamW, learning rate 2×10^{-5}, batch 2, 30 epochs, about 40 s per epoch on the M5 Max.

Table VI is the training log. A first attempt with learning rate 10^{-4} and adversarial weight 1.0 destroyed the images in one epoch (PSNR $39.9 \rightarrow 19.7$) while "fooling" every detector; I report it because it shows why a quality gate is needed. With the settings above, epochs 1–4 pass the gate, and after one epoch R is fooled on every development image and E on all but one; from epoch 5 the quality drifts away and never comes back, so the gate selects epoch 2 (80 s of training).

Table VII is the result on the untouched 100 test images, evaluated with the unchanged Milestone 3 scripts. The reconstruction alone, with no PGD and no latent optimisation, goes from a conditional ASR of $0.01/0.04/0.01/0.15$ (E/R/D/S) to $0.97/0.92/0.05/0.69$ at PSNR 35.1 and LPIPS 0.018, in 2.4 s per image. S was never in the training loss and is still fooled 69% of the time. D (DeiT) is not fooled at all, although it was in the loss. Plugging the fine-tuned branch into the full pipeline raises the official configuration from $0.77/0.39/0.90$ to $1.00/0.58/1.00$ on R/D/S, and under JPEG 75 the ensemble configuration goes from $0.62/0.06/0.09$ to $0.88/0.20/0.23$ on R/D/S, at a cost of about 2 dB PSNR.

D. Limits and what the full-scale version would need

The demo is small: 150 training images from one generator, three detectors in the loss, no compression views. The failure on D says that a decoder trained against two CNNs and one transformer does not transfer to the other transformer, so the full-scale version needs transformer and CLIP detectors in the loss, not just more images. The training is also unstable after a few epochs; a full version would use a lower learning rate for the encoder trunk than for the zero-convolutions, and a held-out quality gate like the one used here. Finally, Table VIII in the next section shows how this attacker fares against the defence I propose: the decoder does not overshoot (AUC 0.74 on E instead of 0.00), so the upper-bound test is blind to it, but the 2-of-4 vote still flags 96% of its images because D and S are not fooled. The arms race continues on the detector axis, not the image axis.

TABLE I
REAL-VS-AI MARGIN m ($\mathrm{LOGIT}_{\mathrm{REAL}} - \mathrm{LOGIT}_{\mathrm{AI}}$) ON LOSSLESS PNG. REAL: 100 IMAGENETTE PHOTOGRAPHS. ATTACKED: SD 1.5 IMAGES CAUGHT BEFORE THE ATTACK (DENOMINATOR OF M3). ">MAX" IS THE FRACTION OF ATTACKED IMAGES WHOSE MARGIN EXCEEDS THE LARGEST REAL-IMAGE MARGIN.

	Real			Official		ENS		CAP-ENS	
Det.	min	med	max	med	>max	med	>max	med	>max
E	0.2	11.0	46.9	153.0	0.99	155.0	0.97	20.7	0.09
R	-0.2	9.0	17.2	1.3	0.00	26.4	0.90	9.7	0.04
D	-0.5	6.1	9.8	-0.8	0.00	9.1	0.03	2.0	0.00
S	5.8	10.5	11.3	5.5	0.00	6.1	0.00	-0.4	0.00

TABLE II
DEFENCE 1, TWO-SIDED TEST ON LOSSLESS PNG. UPPER BOUND U = LARGEST MARGIN OF THE CALIBRATION REAL IMAGES (TWO-FOLD CROSS-FITTING ON 100 REAL IMAGES). CELLS: CONDITIONAL ASR BEFORE $\rightarrow$ AFTER ADDING THE RULE $m > U \Rightarrow$ AI. LAST ROW: REAL-IMAGE FALSE POSITIVE RATE BEFORE $\rightarrow$ AFTER.

Attack	E	R	D	S
PGD-8	1.00→0.00	0.80→0.80	0.38→0.38	0.99→0.99
Official	1.00→0.01	0.77→0.77	0.39→0.39	0.90→0.90
ENS	1.00→0.03	1.00→0.05	1.00→0.94	0.97→0.97
FULL	1.00→0.02	1.00→0.04	1.00→0.93	0.97→0.97
EOT-ENS	1.00→0.00	1.00→0.16	1.00→0.96	0.97→0.97
CAP-E	1.00→0.85	0.13→0.13	0.01→0.01	0.30→0.30
CAP-ENS	1.00→0.85	1.00→0.87	0.66→0.66	0.44→0.44
Real FPR	0.00→0.02	0.01→0.07	0.01→0.02	0.00→0.03

IV. ADVERSARIAL ROLE REVERSAL: DEFENDING AGAINST OUR OWN ATTACK

A. What the attacker knows

If I switch sides, I keep three facts from the attack work. (i) The perturbation that reaches the saved file is a pixel-space PGD perturbation. (ii) The attack overshoots: attacked images are "more real than real" (Table I). (iii) The perturbation does not survive JPEG re-compression, but the released detectors do not survive it either. From these facts I tested three defences. All three keep the four released detectors unchanged. None retrains a model. All numbers below are recomputed from the archived Milestone 3 measurements (100 SD 1.5 images, 100 Imagenette real photographs, four detectors, six post-processing conditions) with the scripts listed in Section IV-G.

Table I shows the problem the attacker created for itself. Real photographs have a bounded margin: on E between 0.2 and 46.9, on S between 5.8 and 11.3. The official attack pushes E to a median of 153; the ensemble pushes R to 26.4 where real photographs never go above 17.2. A detector that only asks "is the margin above zero?" lets these through. A detector that also asks "is the margin inside the range of real photographs?" does not.

B. Defence 1: two-sided score test

The rule is: predict AI if $m < 0$ (the original rule) or $m > U$, where U is the largest margin among a set of real photographs. Nothing else changes.

To set U without leakage I split the 100 real test images into two folds by image index. U is fitted on one fold, the false

TABLE III
DEFENCE 2, PRE-PROCESS THEN DETECT WITH THE *unchanged* THRESHOLD. ATTACK = ENS (STRONGEST M2/M3 VARIANT). LEFT: FRACTION OF THE 100 ATTACKED IMAGES FLAGGED AS AI. RIGHT: REAL-IMAGE FALSE POSITIVE RATE UNDER THE SAME PRE-PROCESSING. "SYS" = FLAG IF ANY OF THE FOUR DETECTORS FLAGS.

	Attacked flagged (ENS)					Real FPR				
Pre-proc.	E	R	D	S	Sys	E	R	D	S	Sys
none (PNG)	0.00	0.00	0.00	0.03	0.03	0.00	0.01	0.01	0.00	0.02
JPEG 95	0.00	0.00	0.05	0.24	0.24	0.03	0.07	0.06	0.24	0.30
JPEG 75	0.07	0.33	0.93	0.88	0.97	0.11	0.27	0.30	0.63	0.74
JPEG 50	0.37	0.66	0.98	0.96	1.00	0.27	0.46	0.43	0.87	0.91
resize 0.75	0.99	0.89	0.95	0.38	1.00	0.80	0.80	0.93	0.15	1.00
resize 0.5	0.98	0.94	0.99	0.31	1.00	0.96	0.97	0.98	0.24	1.00

positive rate is measured on the other fold, and the attacked images never enter the fit. The two folds are swapped and the numbers are averaged.

Table II gives the result on lossless PNG. On E the rule removes almost the whole attack for every variant that does not cap its score: conditional ASR drops from 1.00 to 0.00–0.03. On R it removes the ensemble attack ($1.00 \rightarrow 0.05$). The cost is small: real FPR rises from 0.00 to 0.02 on E and from 0.01 to 0.07 on R. On D and S the rule does nothing, because the attack lands inside the real range there (Table I). The two capped variants (CAP-E, CAP-ENS) were built in Milestone 3 exactly to stay under U, and the table shows they do: only 15% are caught back. The price they pay is visible in the same row: their transfer rates were already low (CAP-E: R 0.13, D 0.01, S 0.30).

C. Defence 2: pre-process, then detect

The second idea is the classic one [6], [7]: re-encode every incoming image as JPEG, or down-scale it, before the detector sees it. Milestone 3 already measured that the attack does not survive this. Table III shows the defender's view for the ENS attack. JPEG 75 makes D flag 93% of attacked images and S 88%. But the right half of the table shows why this is not a free lunch: the released detectors were trained on lossless images, and after JPEG 75 they flag 27–63% of real photographs too. Down-scaling is worse: E, R and D call almost every real photograph AI. So with the thresholds unchanged, pre-processing does not separate attacked images from real ones; it mostly breaks the detector. I report this as a negative result because it is the first thing most people would try.

D. Defence 3: calibrated two-sided thresholds with a vote

Defence 2 fails because the thresholds were not set for the pre-processed input. The fix is cheap: for each detector and each pre-processing condition, set a lower bound L (the 5% quantile of real-image margins under that condition) and an upper bound U (their maximum), and predict AI if $m < L$ or $m > U$. Then combine the four detectors with a 2-of-4 vote so that a single noisy detector cannot drive the false positive rate. Calibration again uses two-fold cross-fitting on the real images only.

Table IV is the main result. With no pre-processing at all, the vote flags 99% of the attacked images for every un-capped variant (PGD-8, official, ENS, FULL, EOT-ENS), 100% of CAP-E and 59% of CAP-ENS, while flagging 100% of un-attacked generated images and only 4% of real photographs. Table V shows where the votes come from: E and S catch nearly everything, R catches the ensemble variants, and D contributes little. Adding JPEG or resizing before detection makes every column worse (ENS: 0.99 on PNG, 0.51 after JPEG 75), because the detectors lose their own signal under compression faster than the attack does. The practical conclusion is the opposite of the usual advice: do not pre-process; calibrate.

E. What an adaptive attacker does next, and why the defender still wins

The capped attacks are the adaptive response [8]: stop the PGD step once the surrogate's margin reaches the median of real photographs. CAP-ENS is the only variant that gets through Defence 3 in a meaningful fraction (41%). But Table II shows what capping costs before any defence is applied: its transfer success on S falls to 0.44 and on D to 0.66, against 0.97 and 1.00 for the un-capped ensemble. In other words, the defender forces the attacker to choose between being caught by the upper bound and losing transfer. Because the attacker only sees the surrogate's score and not the defender's $[L, U]$ interval on the black-box detectors, it cannot aim for the middle of the real range on all four at once. That is why the vote works: an image that is capped correctly for one detector is usually outside the interval for another.

The fine-tuned decoder of Section III is a second adaptive attacker, and a stronger one, because it was trained with a margin loss and does not overshoot at all. Table VIII runs the same defence against it. The upper-bound test of Defence 1 catches none of its images (AUC on E is 0.74, not 0.00; the scores sit inside the real range). Detector E alone, with calibrated thresholds, flags only 17%. But the fine-tuned reconstruction never fooled D, and its S scores fall below the calibrated lower bound, so the 2-of-4 vote still flags 96% of reconstruction-only images and 90–99% when the decoder is combined with the pipeline. What the defender should take from this: the vote's strength is not any single threshold but the detectors the attacker did not, or could not, train against.

F. Limits

Three limits should be stated. The calibration set is small (50 real images per fold), so the upper bound U is noisy; with more real images it would be set at a quantile instead of the maximum. The 2-of-4 vote assumes the defender owns all four detectors. And an attacker who knows $[L, U]$ for every detector, and who has white-box access to all of them, can target the interval directly; the defence raises the cost of the attack, it does not make it impossible.

G. Demo

The defence is implemented in three scripts that run in a few seconds on a laptop and need only the two CSV files of archived measurements:

TABLE IV
DEFENCE 3, CALIBRATED TWO-SIDED THRESHOLDS (L = 5% QUANTILE, U = MAX OF CALIBRATION REAL IMAGES, TWO-FOLD CROSS-FITTING) AND A 2-OF-4 VOTE. CELLS: FRACTION OF THE 100 ATTACKED IMAGES FLAGGED AS AI. "CLEAN": UN-ATTACKED GENERATED IMAGES FLAGGED. "REAL FPR": HELD-OUT REAL PHOTOGRAPHS FLAGGED. ROWS: PRE-PROCESSING APPLIED BEFORE DETECTION (CALIBRATION IS REPEATED UNDER THE SAME PRE-PROCESSING).

Pre-proc.	PGD-8	Official	ENS	FULL	EOT-ENS	CAP-E	CAP-ENS	Clean	Real FPR
none (PNG)	0.99	0.99	0.99	0.99	0.99	1.00	0.59	1.00	0.04
JPEG 95	0.75	0.90	0.96	0.95	0.88	0.89	0.73	0.95	0.05
JPEG 75	0.78	0.79	0.51	0.54	0.69	0.78	0.74	0.86	0.06
JPEG 50	0.71	0.69	0.56	0.50	0.49	0.81	0.80	0.85	0.07
resize 0.75	0.41	0.38	0.42	0.40	0.41	0.45	0.41	0.48	0.03
resize 0.5	0.12	0.10	0.12	0.14	0.12	0.14	0.13	0.12	0.07

TABLE V
DEFENCE 3 ON LOSSLESS PNG, PER DETECTOR (SAME THRESHOLDS AS TABLE IV). FRACTION OF ATTACKED IMAGES FLAGGED; LAST ROW REAL FPR.

Attack	E	R	D	S	2-of-4
PGD-8	1.00	0.56	0.78	0.98	0.99
Official	0.99	0.58	0.80	0.93	0.99
ENS	0.97	0.96	0.06	0.89	0.99
FULL	0.98	0.97	0.07	0.88	0.99
EOT-ENS	1.00	0.86	0.04	0.88	0.99
CAP-E	0.19	0.93	1.00	1.00	1.00
CAP-ENS	0.15	0.17	0.41	1.00	0.59
Real FPR	0.08	0.13	0.07	0.10	0.04

TABLE VI
FINE-TUNING LOG ON THE 10 DEVELOPMENT IMAGES (RECONSTRUCTION ONLY, PNG-QUANTISED). "FLAGGED": FRACTION JUDGED AI BY EACH DETECTOR. GATE: PSNR $\geq$ 32 AND LPIPS $\leq$ 0.05. EPOCH 2 WAS SELECTED (LOWEST MEAN FLAGGED AMONG EPOCHS PASSING THE GATE).

Epoch	E	R	D	S	PSNR	LPIPS	Gate
0 (released)	0.8	0.8	1.0	1.0	39.9	0.004	–
1	0.1	0.0	0.9	0.2	35.8	0.020	yes
2 ⋆	0.0	0.0	0.9	0.2	35.8	0.018	yes
3	0.0	0.0	1.0	0.2	34.2	0.020	yes
4	0.0	0.0	1.0	0.3	33.2	0.024	yes
5	0.1	0.0	1.0	0.3	30.8	0.031	no
6	0.1	0.0	1.0	0.2	27.8	0.050	no
8	0.0	0.0	1.0	0.2	23.0	0.114	no
10	0.0	0.1	1.0	0.3	17.0	0.236	no
15	0.1	0.2	1.0	0.2	12.2	0.450	no
20	0.0	0.2	1.0	0.2	11.1	0.513	no
30	0.0	0.1	1.0	0.4	10.3	0.574	no

- `m4_defense_twosided.py` – Defence 1 (Table II);
- `m4_defense_preprocess.py` – Defence 2 (Table III);
- `m4_defense_calibrated.py` – Defence 3 (Tables IV, V and VIII).

The core of Defence 3 is the following rule, applied per detector and condition:

```
L = np.quantile(cal_real_margins, 0.05)
U = np.quantile(cal_real_margins, 1.00) # max
flag = (margin < L) | (margin > U) # per detector
ai = flag.sum(axis=1) >= 2 # 2-of-4 vote
```

All tables in this section are generated by `make_tables.py` from the script outputs, so no number was typed by hand. Code, result files and this report are in the team repository, folder `m4/`: https://github.com/CAVOK1407/FIT5230-Ass1. The fine-tuning scripts of Section III are `StealthDiffusion/m4_finetune_cvae.py` and `m4_run.sh` in the same repository, with the per-image records under `StealthDiffusion/results/m4_cvae/`.

V. PERSONAL REFLECTION AND BACKGROUND

My undergraduate final-year project was a 1D-CNN for ultrasonic defect detection. Two habits from that project shaped this one. First, a small CNN trained on signals will happily latch onto whatever is easiest to separate, so I learned to check *what* a model responds to before claiming *why* a method works. That is why our Milestone 2 was built as switches and ablations rather than as one improved pipeline, and why the first thing I did in Milestone 3 was a reset test (put the latent back and check the file is identical) before trusting any number. Second, defects in ultrasonic signals are a frequency-domain story, and so are generator fingerprints; the synthetic Freq-PGD experiment in Milestone 1 and the attention to the frequency loss in the released training script both come from that background.

The second advantage was hardware and the decision to use it. I run an Apple M5 Max, and after the Colab T4 ran out of memory in Milestone 1 I moved every experiment to local MPS with resumable scripts. This cost some time at the start (a device shim so the CUDA-only code runs on MPS) but it meant that in Week 11 I could run twenty configurations on a hundred images in two days, and in this milestone fine-tune a decoder and re-run three configurations in an evening.

The third is the plan for what comes after the unit. We intend to go deeper into the method (the training-time route of Section III, more detectors, more generators), improve it further, and try to turn the work into a paper. Because of that, from Week 9 I held the assignment to a paper's standard: pre-registered hypotheses, a stated denominator for every rate, numbers recomputed from saved files, and a hostile review of my own draft. The review caught two errors I had made; the pre-registration is what makes the Milestone 3 negative results credible rather than convenient.

Lessons. I left too much to the deadline weeks; the log above is honest about that. The first fine-tuning attempt in this milestone destroyed the images because I did not check the relative size of the loss terms before training, and I only noticed because the script logged PSNR every epoch.

TABLE VII

FINE-TUNED CONTROL BRANCH VS. RELEASED WEIGHTS ON THE 100 SD 1.5 TEST IMAGES (NEVER USED IN TRAINING). CONDITIONAL ASR PER DETECTOR (DENOMINATOR IN PARENTHESES IN THE HEADER), LOSSLESS PNG AND AFTER JPEG 75. DISTORTION IS MEASURED AGAINST THE CLEAN INPUT. "S/IMG": ATTACK TIME PER IMAGE ON THE M5 MAX (MPS). THE FINE-TUNED WEIGHTS COME FROM 2 EPOCHS ON 150 DISJOINT IMAGES (80 S).

	Lossless PNG				JPEG 75							
Configuration	E (94)	R (84)	D (99)	S (98)	E (82)	R (88)	D (99)	S (97)	PSNR	LPIPS	L_∞	s/img
Recon. only, released	0.01	0.04	0.01	0.15	0.04	0.02	0.00	0.00	39.1	0.005	0.172	2.4
Recon. only, fine-tuned	0.97	0.92	0.05	0.69	0.56	0.33	0.01	0.06	35.1	0.018	0.218	2.4
Official, released	1.00	0.77	0.39	0.90	0.91	0.15	0.02	0.06	34.0	0.126	0.183	10.0
Official, fine-tuned	1.00	1.00	0.58	1.00	1.00	0.58	0.04	0.19	31.8	0.145	0.229	10.2
ENS, released	1.00	1.00	1.00	0.97	0.91	0.62	0.06	0.09	33.9	0.126	0.183	11.1
ENS, fine-tuned	1.00	1.00	1.00	1.00	1.00	0.88	0.20	0.23	31.7	0.146	0.229	11.4

TABLE VIII

THE FINE-TUNED ATTACKER AGAINST THE SECTION IV DEFENDER (LOSSLESS PNG). AUC: REAL VS. ATTACKED IMAGES UNDER THE RELEASED DETECTORS (0.5 = INSEPARABLE; 0 = SEPARABLE BECAUSE OF OVERSHOOT). RIGHT: FRACTION FLAGGED BY DEFENCE 3 (CALIBRATED TWO-SIDED THRESHOLDS, 2-OF-4 VOTE), SAME THRESHOLDS AS TABLE IV.

	AUC real vs. attacked				Flagged by Defence 3				
Configuration	E	R	D	S	E	R	D	S	2-of-4
Recon. only, released	1.00	0.99	1.00	1.00	0.99	0.97	1.00	1.00	1.00
Recon. only, fine-tuned	0.74	0.76	1.00	0.99	0.17	0.36	0.98	0.97	0.96
Official, released	0.00	0.84	0.96	0.99	0.99	0.58	0.80	0.93	0.99
Official, fine-tuned	0.00	0.53	0.91	0.97	1.00	0.28	0.55	0.82	0.90
ENS, released	0.01	0.00	0.15	0.98	0.97	0.96	0.06	0.89	0.99
ENS, fine-tuned	0.00	0.00	0.15	0.96	0.99	0.96	0.06	0.78	0.99
Real FPR					0.08	0.13	0.07	0.10	0.04

And the PreDecodeGuard episode taught me that the right answer to a well-written protocol can still be "no" for the part that produces harmful artefacts, and that saying so, with alternatives, is itself analysis worth marks.

VI. CONCLUSION

As the attacker, our team found that the released StealthDiffusion attack lives in a pixel PGD step and overshoots the detector's real-image range. As the defender in this report, I turned the second finding into a defence that keeps the released detectors and only re-calibrates their decision interval: a 2-of-4 vote flags 99% of our attacked images at a 4% false positive rate, and pre-processing turns out to hurt rather than help. As the unconstrained attacker, I moved the detector objective into the training of the Control-VAE control branch; 80 seconds of fine-tuning make the plain reconstruction evade three of four detectors without overshooting, which blinds the upper-bound test but not the vote, because the transformer detector D never fell. Both sides of the report therefore point to the same place: the next move, for either side, is on the detector axis.

REFERENCES

[1] Z. Zhou, K. Sun, Z. Chen, H. Kuang, X. Sun, and R. Ji, "StealthDiffusion: Towards evading diffusion forensic detection through diffusion model," in *Proc. 32nd ACM Int. Conf. Multimedia (MM)*, 2024, arXiv:2408.05669.

[2] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2018.

[3] M. Zhu, H. Chen, Q. Yan, X. Huang, G. Lin, W. Li, Z. Tu, H. Hu, J. Hu, and Y. Wang, "GenImage: A million-scale benchmark for detecting AI-generated image," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.

[4] U. Ojha, Y. Li, and Y. J. Lee, "Towards universal fake image detectors that generalize across generative models," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023.

[5] A. Athalye, L. Engstrom, A. Ilyas, and K. Kwok, "Synthesizing robust adversarial examples," in *Proc. 35th Int. Conf. Machine Learning (ICML)*, 2018.

[6] G. K. Dziugaite, Z. Ghahramani, and D. M. Roy, "A study of the effect of JPG compression on adversarial images," *arXiv preprint arXiv:1608.00853*, 2016.

[7] C. Guo, M. Rana, M. Cissé, and L. van der Maaten, "Countering adversarial images using input transformations," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2018.

[8] N. Carlini and D. Wagner, "Adversarial examples are not easily detected: Bypassing ten detection methods," in *Proc. 10th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2017, pp. 3–14.